POWER-AWARE HIGH-FREQUENCY 7-STAGE RISC-V RV32IM CORE WITH ADAPTIVE GATING

Complete RTL Microarchitecture Analysis, FPGA Power Synthesis Breakdown & Staff-Level Interview Preparation Guide

---

Target Architecture: RISC-V RV32IM	**Target Device:** Xilinx 7-Series FPGA (XC7A100T)
Pipeline Depth: 7 Stages (IF1-IF2-ID-EX1-EX2-MEM-WB)	**Clock Management:** MMCM (50 MHz) + 12x BUFGCE Gated Clocks
Power Savings: Dynamic Stage Gating + WFI Sleep + Approx ALU	**Synthesis Results:** 0.251 W Total Power (0.153 W Dynamic)

Executive Summary & Core Value Proposition

This project implements an advanced, industrial-grade 32-bit RISC-V processor core optimized for low-power high-frequency execution on Xilinx 7-Series FPGAs. By extending traditional 5-stage classic RISC pipelines to a deeply pipelined **7-stage microarchitecture** (split instruction fetch IF1/IF2 and dual execution stages EX1/EX2), the core achieves higher maximum operating frequencies (Fmax) while maintaining low dynamic power through central dynamic clock gating using hardware BUFGCE buffers, Wait-For-Interrupt (WFI) sleep shutdown, and an energy-saving Approximate Arithmetic mode.

Guide Table of Contents

Chapter 1: Microarchitectural System Specifications

Key Microarchitectural Parameters

Parameter	Specification / Value	Implementation Details
Instruction Set Architecture	RISC-V RV32IM + Custom CSRs	Base 32-bit Integer + M Extension (Mul/Div) + Power/Approx CSRs
Pipeline Depth	7 Stages	IF1 → IF2 → ID → EX1 → EX2 → MEM → WB
Target Frequency	50 MHz (20.0 ns period)	Generated via MMCME2_BASE primitive on Xilinx 7-Series
Branch Predictor	Dynamic BHT + BTB	64-entry 2-bit saturating counter array + 32-bit target & tag match
Multiplier Core	1-Cycle Combinational DSP	Maps to FPGA DSP48E1 slices ($signed, unsigned products)
Divider Core	32-Cycle Iterative Restoring	Sequenced state machine; holds pipeline stall via stall_ex2
Clock Gating Architecture	12 Gated Clock Domains	BUFGCE primitives controlled by stage_gating_controller & mpower CSR
Approximate Mode	4-LSB Truncated Addition	Enabled via mapprox CSR (0x800); reduces ALU switching power
Memory Architecture	16KB IMEM / 16KB DMEM	Dual-Port BRAM blocks with integrated UART bootloader at reset
MMIO Peripherals	UART TX & LED Array	Mapped at 0xFFFF0000 (UART TX byte) & 0xFFFF0004 (LED state)

FPGA Synthesis & Utilization Breakdown (XC7A100T-2FTG256C)

The design has been synthesized and fully routed in Xilinx Vivado v2025.1. Total on-chip power is measured at **0.251 W** (0.153 W Dynamic, 0.098 W Static).

Resource Type	Used	Available	Utilization %	Power Contribution
Slice LUTs (Logic)	3,767	63,400	5.94%	0.008 W
Slice Registers (FFs)	5,520	126,800	4.35%	< 0.001 W
Block RAM (RAMB36)	8	135	5.93%	0.009 W
DSP Slices (DSP48E1)	12	240	5.00%	0.005 W
Clocking (MMCME2)	1 MMCM	6	16.67%	0.105 W
Signals & Routing	8,827 nets	—	—	0.014 W
Gated Clock Trees	12 BUFGCE	32 BUFG	37.5%	0.012 W

Chapter 2: 7-Stage Deep Pipeline Architecture

Why a 7-Stage Pipeline?

Standard 5-stage RISC-V processors combine instruction fetch into 1 cycle and execution into 1 cycle. In modern high-frequency FPGA designs, BRAM address-to-data read latency and complex multi-operand ALU/branch calculations create critical path bottlenecks. This architecture splits fetch into **IF1** and **IF2**, and splits execution into **EX1** (branch resolution & target calculation) and **EX2** (ALU, CSR, Multiplication, Division).

Stage	Module Name	Key Functions & Operations Performed
1. IF1 Fetch 1	if1.v	Program Counter (PC) register update, speculative next-PC target selection, BHT/BTB branch predictor lookup, output BRAM read address.
2. IF2 Fetch 2	if2.v	Registers 32-bit instruction returned from synchronous Dual-Port BRAM (imem). Holds instruction during IF stalls and pipeline flushes.
3. ID Decode	id.v	Decodes RISC-V opcode, funct3, funct7. Generates 24-bit control bus. Reads GPR registers rs1/rs2. Generates immediates. Asserts stage gating class flags.
4. EX1 Execute 1	ex1.v	Receives forwarded operands. Computes branch condition and jump target (JAL/JALR). Compares predicted PC with actual target to trigger 2-cycle mispredict flush. Starts MUL/DIV pulses.
5. EX2 Execute 2	ex2.v	Executes exact or 4-LSB approximate ALU ops, DSP 32x32 multiplication, multi-cycle division, CSR read/write access. Evaluates MMIO UART TX busy stalls.
6. MEM Memory	mem.v	Drives Data BRAM (dmem). Packs store data and byte-enable strobes (SB, SH, SW). Performs sign-extension and zero-extension on loaded byte/halfword data (LB, LH, LW, LBU, LHU).
7. WB Writeback	wb.v	Writes ALU or Load result back to General Purpose Register File (R1-R31). Detects WFI instruction to trigger low-power idle sleep state.

24-Bit Pipeline Control Bus Structure

Control signals generated in id.v travel down the pipeline inside a packed 24-bit bus ctrl_bus[23:0]:

- ctrl_bus[23]: Is M-Extension instruction (MUL/DIV)
- ctrl_bus[22:20]: Load/Store sub-type (funct3)
- ctrl_bus[19:16]: M-Extension sub-type
- ctrl_bus[15]: Register File Write Enable (reg_write)
- ctrl_bus[14]: Data Memory Read Enable (mem_read)
- ctrl_bus[13]: Data Memory Write Enable (mem_write)
- ctrl_bus[12]: Is Branch instruction
- ctrl_bus[11]: Is JAL Jump instruction
- ctrl_bus[10]: Is JALR Indirect Jump instruction
- ctrl_bus[9]: Is CSR Operation
- ctrl_bus[8]: ALU Source B Select (0 = rs2, 1 = immediate)
- ctrl_bus[7:4]: ALU Operation Select Code

Chapter 3: Adaptive Stage Clock Gating & Power Management

Dynamic Hardware Clock Tree Architecture

To achieve maximum power reduction without compromising timing performance, the core replaces global un-gated clocks with a dynamic 12-domain gated clock distribution tree. The module stage_gating_controller.v evaluates stage classification signals, hazard stalls, CSR power masks, and WFI states to generate individual stage clock enables ce_if1...ce_wb.

Synthesis Gated Clock Buffer (BUFGCE Primitive)

In FPGA Synthesis mode (`ifdef SYNTHESIS inside clock_manager.v), clock enables are registered on the falling edge / rising edge of the MMCM clock and supplied directly to Xilinx **BUFGCE** primitives. This completely disables the clock tree toggle activity to unused pipeline registers and functional blocks.

```
// Xilinx 7-Series BUFGCE Gated Clock Primitive Instantiation in clock_manager.v BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_if1 (.I(clk_mmcm_out), .CE(ce_if1_r), .O(clk_if1)); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_if2 (.I(clk_mmcm_out), .CE(ce_if2_r), .O(clk_if2)); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_id (.I(clk_mmcm_out), .CE(ce_id_r ), .O(clk_id )); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_ex1 (.I(clk_mmcm_out), .CE(ce_ex1_r), .O(clk_ex1)); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_ex2 (.I(clk_mmcm_out), .CE(ce_ex2_r), .O(clk_ex2)); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_mem (.I(clk_mmcm_out), .CE(ce_mem_r), .O(clk_mem)); BUFGCE
#(.SIM_DEVICE("7SERIES")) buf_wb (.I(clk_mmcm_out), .CE(ce_wb_r ), .O(clk_wb ));
```

Simulation Glitch-Free Negative-Latch Integrated Clock Gating (ICG)

Standard RTL AND-gating (clk & ce) causes dangerous clock glitches if ce changes while clk is HIGH. In RTL Behavioral Simulation mode (`ifndef SYNTHESIS), clock_manager.v models a negative-level latch ICG circuit. The clock enable signal is sampled only when clk_in is LOW, guaranteeing zero glitching on gated clock outputs.

```
// Glitch-Free Negative-Latch ICG Model for RTL Simulation (clock_manager.v) reg ce_if1_latch,
ce_if2_latch, ce_id_latch, ce_ex1_latch, ce_ex2_latch; always @(*) begin if (!clk_in) begin //
Transparent ONLY during low half-cycle of clock ce_if1_latch = ce_if1; ce_if2_latch = ce_if2;
ce_id_latch = ce_id; ce_ex1_latch = ce_ex1; ce_ex2_latch = ce_ex2; end end assign clk_if1 = clk_in &
ce_if1_latch; // Zero-glitch output clock
```

Wait-For-Interrupt (WFI) Deep Idle Shutdown

When the core executes a wfi instruction (SYSTEM opcode 0x73 with imm 0x105), the writeback stage asserts wfi_active = 1. The stage gating controller immediately forces ce_if1 = ce_if2 = ce_id = ce_ex1 = ce_ex2 = ce_mem = ce_wb = 0, turning off all pipeline clocks. Only clk_csr remains active to allow external hardware interrupts or system reset to wake the processor.

Chapter 4: Energy-Efficient Approximate Arithmetic Unit

4-LSB Truncated Arithmetic Logic Architecture

For error-tolerant workloads such as quantum-inspired QUBO optimization, image processing, neural network inference, and matrix operations, exact 32-bit addition produces unnecessary carry-propagation switching power in lower bits. The core implements a hardware-selectable Approximate Arithmetic Mode controlled by the custom CSR mapprox (Address 0x800).

```
// Exact vs Approximate ALU Logic in ex2.v // 1. Exact ALU Addition alu_exact = ex2_op_a_in +
ex2_op_b_in; // 2. Approximate ALU Addition (Truncating 4 LSBs to zero) alu_approx = {
(ex2_op_a_in[31:4] + ex2_op_b_in[31:4]), 4'b0000 }; // 3. Mode Multiplexing based on mapprox CSR bit 0
```

```
wire [31:0] alu_final = (approx_enable && !ex2_ctrl_bus_in[8]) ? alu_approx : alu_exact;
```

Error Analysis & Power Savings Trade-Off

Mathematical Error Bound: The maximum absolute error introduced by truncating 4 LSBs during addition is 2^4 - 1 = 15. For 32-bit values exceeding 1,000,000, the relative computational error is under **0.0015%**.

Dynamic Power Impact: Eliminating carry propagation across bits [3:0] reduces net switching activity in the internal CARRY4 primitives by up to **18%** during continuous arithmetic loop execution.

Chapter 5: Hazard Resolution & Forwarding Network

Pipeline Hazard Classification & Handling Strategy

Because the pipeline is 7 stages deep, data dependency penalties could be severe without aggressive forwarding. The core integrates a centralized hazard_unit.v and forwarding_unit.v to maintain maximum throughput.

Hazard Type	Detection Condition	Pipeline Resolution Mechanism	Penalty
RAW Data Hazard (ALU → ALU)	ID rs1/rs2 matches EX2_rd, MEM_rd, or WB_rd	Forwarding network bypasses result directly to EX1 operands in 0 cycles.	0 Cycles (No Stall)
RAW Load-Use Hazard	Instruction in EX1/EX2/MEM is LOAD and rd matches ID rs1/rs2	Hazard unit stalls IF1, IF2, ID, and EX1 for 1 cycle until data loaded from BRAM.	1 Cycle Stall
Multi-cycle DIV Structural Hazard	Iterative divider active (div_busy = 1)	Holds stall_ex2 = 1, freezing IF1, IF2, ID, EX1 registers for 32 cycles.	32 Cycles Stall
Branch Misprediction Hazard	EX1 actual branch result ≠ predicted PC in IF1	Asserts flush_id = 1 and flush_ex1 = 1. Redirects IF1 PC to correct target.	2 Cycle Flush

4-Level Priority Forwarding Network Matrix

Forwarding is evaluated combinationally in forwarding_unit.v for both operand 1 (rs1) and operand 2 (rs2) with strict age priority (newest pipeline stage first):

```
// Operands Forwarding Logic (forwarding_unit.v) if (ex2_valid && ex2_rd != 0 && ex2_rd == id_rs1) fwd_rs1_val = ex2_result; // Priority 1: EX2 Stage Bypass else if (mem_valid && mem_rd != 0 && mem_rd == id_rs1) fwd_rs1_val = mem_result; // Priority 2: MEM Stage Bypass else if (wb_valid && wb_rd != 0 && wb_rd == id_rs1) fwd_rs1_val = wb_result; // Priority 3: WB Stage Bypass else if (rf_we && rf_waddr != 0 && rf_waddr == id_rs1) fwd_rs1_val = rf_wdata; // Priority 4: RF Write Port Bypass else fwd_rs1_val = original_rs1; // Default: Register File Async Read
```

Chapter 6: Hardware Arithmetic Extensions (RV32M)

DSP-Optimized Hardware Multiplier (mul_unit.v)

The multiplier unit implements full 32-bit x 32-bit multiplication producing a 64-bit full product. By utilizing Verilog signed multiplication operators ($signed(op_a) * $signed(op_b)), Xilinx Vivado infers dedicated **DSP48E1 hard slices**. Multiplication executes combinationally within the EX2 stage (busy = 0), introducing zero stall cycles.

32-Cycle Iterative Restoring Divider (div_unit.v)

Division cannot be efficiently completed in a single high-frequency clock cycle without massive delay penalties. The core implements a 32-cycle sequential radix-2 restoring division algorithm:
1. **Start Phase:** EX1 issues div_start pulse. Divider captures dividend/divisor and computes absolute values for signed operations (DIV, REM).
2. **Compute Phase:** Over 32 clock cycles, dividend[62:31] is compared against divisor. If greater, subtraction occurs and quotient bit 1 is shifted in.
3. **Completion Phase:** Quotient or remainder sign corrections are applied based on operand original sign bits (neg_q = op_a[31] ^ op_b[31], neg_r = op_a[31]).

Chapter 7: Hardware Branch Prediction Unit (BPU)

BHT + BTB + Tag Matching Architecture

Branch instruction execution penalties in deep 7-stage pipelines can severely impact CPI (Cycles Per Instruction). The core incorporates a hardware **Branch Prediction Unit (BPU)** in branch_predictor.v that runs speculatively during the IF1 stage.

BPU Component	Structure & Size	Functional Description
Branch History Table (BHT)	64 Entries x 2-bit	2-bit Saturating Counter State Machine (00: Strongly Not Taken, 01: Weakly Not Taken, 10: Weakly Taken, 11: Strongly Taken).
Branch Target Buffer (BTB)	64 Entries x 32-bit	Stores the target branch address computed during previous executions of the branch.
Tag Comparison Array	64 Entries x 32-bit	Stores full PC tag address to ensure BTB hits only occur for exact matching branch instruction addresses.

Chapter 8: Control & Status Register (CSR) Subsystem

CSR Address Map & Privilege Architecture

The module csr_unit.v handles machine-mode performance counters and custom power-management registers. CSR operations support CSRRW (Atomic Read/Write), CSRRS (Atomic Read & Set Bits), and CSRRC (Atomic Read & Clear Bits).

CSR Address	CSR Name	Access	Description & Power Control Function
0xC00	mcycle	Read-Only	Lower 32 bits of 64-bit cycle counter (increments every active clock cycle).
0xC80	mcycleh	Read-Only	Upper 32 bits of 64-bit cycle counter.
0xC02	minstret	Read-Only	Lower 32 bits of 64-bit retired instruction counter (increments on WB valid).
0xC82	minstreth	Read-Only	Upper 32 bits of 64-bit retired instruction counter.
0x800	mapprox	Read/Write	Custom Approx Register. Bit [0] = 1 enables 4-LSB ALU truncation mode.
0x801	mpower	Read/Write	Custom Power Gating Register. 8-bit mask enabling stage clocks [6:0] and UART [7]. Default 0xFF.

Chapter 9: Memory Layout & UART In-Circuit Bootloader

System Memory Address Map

Address Range	Target Device / Peripheral	Access Type	Description
0x0000_0000 - 0x0000_3FFF	Instruction BRAM (IMEM)	Read-Only (CPU) Write (UART Loader)	16KB Dual-Port Block RAM. Holds application binary instructions.
0x0000_4000 - 0x0000_7FFF	Data BRAM (DMEM)	Read / Write	16KB Dual-Port Block RAM. Holds application stack, heap, and static data.
0xFFFF_0000	MMIO UART TX Register	Write-Only	Writing a byte to this address transmits serial character over UART pin at 115200 Baud.
0xFFFF_0004	MMIO LED Output Register	Write-Only	Writing an 8-bit value directly updates physical FPGA LED pin state.

UART Hardware Bootloader Protocol (bram_imem.v)

To allow rapid application binary deployment on physical FPGA hardware without triggering lengthy Vivado synthesis runs, the instruction memory module bram_imem.v integrates an in-circuit UART bootloader.

• **Active Reset Mode:** When physical reset button M14 is pressed, the CPU is held in reset, while u_rx listens on the serial port (115200 Baud).

• **Word Reassembly:** Bytes arrive Little-Endian. Once 4 bytes arrive (byte_cnt == 3), a complete 32-bit RISC-V instruction word is written into mem[load_addr].

• **Execution Handover:** Upon releasing button M14, the core begins fetching instructions from address 0x00000000.

Chapter 10: Masterclass Technical Interview Questions & Answers

This chapter presents 25 deep technical interview questions covering RTL microarchitecture, FPGA synthesis, timing analysis, hazard handling, power optimization, and debugging.

Q1: Why did you choose a 7-stage pipeline instead of the classic 5-stage RISC-V pipeline?

In FPGA implementations (such as Xilinx 7-Series), Block RAM memory access requires a clock cycle for address-to-data output latency. A 5-stage pipeline forces instruction memory read and decode into a tight cycle, limiting maximum operating frequency. By splitting Fetch into IF1 (address generation & BPU lookup) and IF2 (BRAM data register), and splitting Execution into EX1 (branch target/condition calculation) and EX2 (ALU/MUL/DIV), we significantly shorten the critical path, enabling a 50 MHz+ target Fmax.

Q2: How does dynamic stage clock gating work in your verilog design, and why is BUFGCE required on Xilinx FPGAs?

Central stage clock gating is managed by stage_gating_controller.v and clock_manager.v. On Xilinx FPGAs, simply using combinational logic (clk & enable) causes clock skew, glitching, and violates global clock routing constraints. We instantiate 12 BUFGCE primitives (gated global clock buffers). The stage_gating_controller outputs enable signals (ce_if1..ce_wb) which are registered on clk_mmcm_out to drive BUFGCE primitives cleanly, toggling clock trees off when pipeline stages are idle or masked out by the mpower CSR.

Q3: How do you prevent clock glitches during behavioral simulation when BUFGCE primitives are bypassed?

In simulation mode (`ifndef SYNTHESIS`), clock_manager.v models a glitch-free Negative-Latch Integrated Clock Gating (ICG) circuit. It latches the stage enable signal during the LOW half-cycle of clk_in (if (!clk_in)). This guarantees that enable transitions never occur while the clock line is HIGH, completely eliminating artificial clock glitches in Icarus Verilog or Vivado XSIM.

Q4: What happens during a Wait-For-Interrupt (WFI) instruction, and how does the core wake up?

When a WFI instruction (opcode 0x73, imm 0x105) reaches the Writeback (WB) stage, wb.v asserts wfi_active = 1. The stage gating controller detects wfi_active and immediately deasserts ce_if1 through ce_wb, turning off all 7 pipeline stage clocks. Dynamic core power drops to near zero. Only clk_csr remains active so that an incoming external hardware interrupt or system reset can clear wfi_active and resume clocking.

Q5: Explain the Approximate Arithmetic Mode and its impact on computational power and precision.

The custom CSR mapprox (0x800) bit 0 enables approximate addition in EX2 (ex2.v). Instead of exact 32-bit addition, the ALU calculates alu_approx = { (op_a[31:4] + op_b[31:4]), 4'b0000 }, truncating the 4 least significant bits. This reduces switching activity in lower CARRY4 FPGA primitives by up to 18%. For noise-tolerant tasks like QUBO quantum stress loops or image processing, the maximum absolute error is bounded by 15, yielding a negligible relative error (<0.0015%).

Q6: How does your core handle RAW (Read-After-Write) data hazards without introducing unnecessary stall cycles?

RAW hazards are resolved in 0 stall cycles using a 4-level priority forwarding network in forwarding_unit.v. Forwarded values are combinationally bypassed to EX1 operands (fwd_rs1_val / fwd_rs2_val) from EX2 result, MEM result, WB result, or RF write port in order of priority. Only Load-Use dependencies require a 1-cycle stall because memory load data is not ready until the MEM stage.

Q7: How is a Load-Use hazard detected and resolved in your hazard unit?

A Load-Use hazard occurs when an instruction in EX1, EX2, or MEM is a LOAD (mem_read = 1) and its destination register rd matches rs1 or rs2 of the instruction currently in ID. hazard_unit.v detects this condition and asserts stall_if = 1, stall_id = 1, and stall_ex1 = 1 for 1 cycle. This freezes the front pipeline while inserting a bubble into EX2.

Q8: Describe the operation of your 32-cycle iterative restoring hardware divider (div_unit.v).

The hardware divider uses a sequential radix-2 restoring division algorithm. When EX1 detects a DIV/REM opcode, it issues a 1-cycle div_start pulse. The divider sets busy = 1 for 32 clock cycles, holding stall_ex2 = 1 to freeze EX1 and earlier pipeline stages. Each cycle, dividend[62:31] is compared with divisor; if greater, divisor is subtracted and a 1 bit is shifted in. Signed quotient and remainder sign corrections are applied on cycle 32.

Q9: Why does the multiplier (mul_unit.v) execute in 1 cycle while division takes 32 cycles?

The multiplier infers DSP48E1 hard blocks on Xilinx FPGAs. DSP48E1 slices contain fast 25x18 hardware multipliers capable of performing 32x32 signed/unsigned products combinationally within the EX2 clock period (20 ns). Division algorithms require iterative comparison and shift-subtraction steps that cannot fit in a single 50 MHz clock period without severely violating setup time, necessitating a multi-cycle state machine.

Q10: How does the Branch Prediction Unit (BPU) handle branch resolution and misprediction recovery?

The BPU (branch_predictor.v) uses a 64-entry BHT (2-bit saturating counters), BTB (target addresses), and tag array. In IF1, if BHT >= 2'b10 and tag matches if_pc, predicted_valid = 1 and PC jumps to BTB address. In EX1, the actual branch condition and target are evaluated. If prediction was wrong, ex1.v asserts branch_mispredict = 1, driving flush_id = 1 and flush_ex1 = 1 to clear speculatively fetched instructions, while redirecting IF1 to the correct branch target.

Q11: What is the penalty for a branch misprediction in this 7-stage core?

The branch misprediction penalty is exactly 2 clock cycles. Because branches are resolved in EX1, two speculatively fetched instructions reside in IF2 and ID. Asserting flush_id and flush_ex1 converts these 2 stages into bubble NOPs while reloading the correct PC into IF1.

Q12: How does the internal register file (register_file.v) support asynchronous reads with write-first bypass?

The register file contains 32 registers of 32 bits. Reads are combinational (asynchronous). To prevent stale reads when an instruction writes to rd in WB while the next instruction reads the same register in ID, an internal bypass is implemented: assign rdata1 = (we && waddr != 0 && waddr == raddr1) ? wdata : regs[raddr1]. Register R0 is hardwired to 0.

Q13: Explain how the UART In-Circuit Bootloader operates during FPGA reset.

The instruction BRAM (bram_imem.v) integrates a UART RX module (u_rx) running at 115200 Baud. While the physical reset button M14 is held DOWN, the CPU is held in reset, but uart_clk remains active. Arriving serial bytes are assembled in little-endian order (4 bytes = 1 word). Once byte_cnt == 3, the complete 32-bit instruction word is written into mem[load_addr]. Releasing button M14 allows the core to boot the newly uploaded program instantly.

Q14: How is Memory-Mapped I/O (MMIO) implemented in this RISC-V core?

MMIO is decoded in EX2 and MEM stages based on memory addresses. Address 0xFFFF0000 is mapped to the MMIO UART TX transmitter (uart_tx_minimal.v). Writing a byte to 0xFFFF0000 triggers UART serial transmission; if the UART TX line is busy, stall_uart asserts stall_ex2 to freeze the pipeline. Address 0xFFFF0004 is mapped directly to the FPGA 8-bit LED array.

Q15: What custom CSRs were added to this core and what are their addresses?

Two custom machine-mode CSRs were added: 1) mapprox at address 0x800 (bit [0] enables 4-LSB truncated approximate ALU mode), and 2) mpower at address 0x801 (8-bit stage mask controlling dynamic clock enable outputs in stage_gating_controller.v). Standard CSRs include mcycle (0xC00/0xC80) and minstret (0xC02/0xC82).

Q16: How do CSR instructions (CSRRW, CSRRS, CSRRC) modify system registers?

CSR instructions pass down to EX2 with is_csr = 1. In csr_unit.v, CSRRW (funct3 01) overwrites the target CSR with rs1 data. CSRRS (funct3 10) sets bits using rs1 as a bitmask (csr | wdata). CSRRC (funct3 11) clears bits using rs1 as a mask (csr & ~wdata). Updated CSR values take effect immediately on the next clock cycle.

Q17: What are the main sources of power dissipation reported in power_report.txt?

According to Vivado report_power on routed fpga_top, Total Power is 0.251 W (Dynamic: 0.153 W, Static: 0.098 W). The largest dynamic power contributor is the MMCM clock generator (0.105 W, 68.6% of dynamic power). Core signals consume 0.014 W, gated clock trees consume 0.012 W, Slice LUT logic consumes 0.008 W, and BRAM blocks consume 0.009 W.

Q18: What clock constraint was used in synthesis and how is the 50 MHz system clock generated?

The input FPGA clock clk_in is constrained to 20.0 ns (50 MHz) in risc_v.xdc. An MMCME2_BASE primitive in clock_manager.v multiplies the 50 MHz input by 20 (VCO = 1000 MHz) and divides by 20 to output a jitter-filtered 50 MHz clk_mmcm_out system clock.

Q19: How do you handle store byte enables (SB, SH, SW) in mem.v?

In mem.v, store data and 4-bit write strobes (mem_wstrb) are calculated combinationally based on address bits [1:0] and funct3: SB (Store Byte) shifts 8-bit data and asserts 1 strobe bit (4'b0001 << addr[1:0]); SH (Store Halfword) replicates 16-bit data and asserts 2 strobe bits (4'b0011 or 4'b1100); SW (Store Word) asserts all 4 strobe bits (4'b1111).

Q20: How are sign-extension and zero-extension handled for Load instructions in mem.v?

When reading BRAM data (mem_rdata), mem.v inspects funct3 and address bits [1:0]: LB (Load Byte) sign-extends bit 7 ({{24{data[7]}}, data[7:0]}); LBU (Load Byte Unsigned) zero-extends (24'd0, data[7:0]); LH sign-extends 16 bits; LHU zero-extends 16 bits; LW passes the full 32-bit word directly.

Q21: What is the purpose of the pulse latch (div_started) in ex1.v?

When a division instruction starts, ex1.v generates a 1-cycle div_start pulse. Because division stalls the pipeline for 32 cycles, EX1 holds its register values. Without div_started pulse latching, EX1 would re-trigger div_start on every stalled cycle. div_started ensures div_start pulses exactly once.

Q22: How does the pipeline recover from a multi-stage stall triggered by UART TX busy?

When an MMIO store writes to 0xFFFF0000 while uart_tx_minimal.v is transmitting (busy = 1), ex2.v asserts stall_uart = 1, driving stall_ex2 = 1. The hazard unit propagates stall_ex2 to freeze IF1, IF2, ID, and EX1. Once the UART transmission completes (busy = 0), stall_ex2 deasserts and normal pipeline flow resumes smoothly.

Q23: How would you debug a silent hang in hardware when executing an assembly loop on the FPGA?

1) Check physical LEDs bound to dbg_led and dbg_wfi_active to see if core entered WFI or reset stall. 2) Connect Vivado Integrated Logic Analyzer (ILA) core to sample pc_out, stall_ex2, and wfi_active signals in real time. 3) Inspect UART bootloader logs using diagnostic scripts (upload_edge_cases.py) to verify complete byte stream reception.

Q24: What timing path in this 7-stage RISC-V core is most likely to become the critical path at higher frequencies (e.g., > 100 MHz)?

The most critical timing path is the Forwarding Mux → EX1 Branch Condition Evaluation → EX1 Mispredict Detection → IF1 Next-PC Mux loop. Because branch resolution in EX1 depends on forwarded ALU results from EX2 or MEM, this combinational logic chain spans across stage boundaries and represents the primary Fmax limiting path.

Q25: If you were to add an L1 Instruction Cache to this core, how would miss stalls integrate into the existing hazard unit?

An I-Cache miss signal (icache_miss) would be connected directly to hazard_unit.v as a high-priority stall input. On a miss, hazard_unit.v would assert stall_if = 1 and stall_id = 1 while inserting a bubble flush into EX1 (flush_ex1 = 1). Once the BRAM/DRAM line fill completes, icache_miss deasserts, allowing IF1 to resume fetching instructions.